

人工智能课程——编程题练习

sklearn / Keras 调库实战

2026 年 6 月 28 日

说明：

- 本卷共 6 道编程题，满分 100 分。
- 考察范围：线性回归、逻辑斯蒂分类、K-Means、朴素贝叶斯、全连接网络、CNN。
- 允许调用：sklearn、TensorFlow/Keras、NumPy、Pandas、Matplotlib。
- 每题要求写出完整可运行的 Python 代码，并给出关键输出结果。
- 每道题后附有参考答案，建议先独立完成再核对。

第一题：线性回归——房价预测（15 分）

使用 sklearn 内置的加州房价数据集（`fetch_california_housing`）。

要求写出完整代码实现以下步骤：

- (1) 加载数据，取 MedInc（收入中位数）作为特征 X ，目标为房价 y 。（2 分）
- (2) 划分训练集/测试集（7:3，`random_state=42`）。（2 分）
- (3) 训练线性回归模型，输出学到的权重 w 、偏置 b 。（3 分）
- (4) 计算训练集和测试集的 R^2 ，判断是否存在过拟合或欠拟合。（4 分）
- (5) 绘制散点图（测试集用不同颜色）+ 回归直线。（4 分）

【参考答案】

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import fetch_california_housing
4 from sklearn.linear_model import LinearRegression
5 from sklearn.model_selection import train_test_split
6
```

```

7 # (1) 加载: MedInc 是第0列
8 data = fetch_california_housing()
9 X = data.data[:, 0:1] # 取 MedInc 列, 保持二维
10 y = data.target
11
12 # (2) 划分
13 X_train, X_test, y_train, y_test = train_test_split(
14     X, y, test_size=0.3, random_state=42)
15
16 # (3) 训练
17 model = LinearRegression()
18 model.fit(X_train, y_train)
19 print(f"w = {model.coef_[0]:.4f}") # w 0.4264
20 print(f"b = {model.intercept_:.4f}") # b 0.4473
21
22 # (4) R^2 评估
23 r2_train = model.score(X_train, y_train)
24 r2_test = model.score(X_test, y_test)
25 print(f"训练 R^2 = {r2_train:.4f}") # 0.478
26 print(f"测试 R^2 = {r2_test:.4f}") # 0.466
27 # 判断: 两者接近且都不高 → 欠拟合(单特征不足以解释房价)
28
29 # (5) 绘图
30 plt.scatter(X_train, y_train, c='blue', s=2, alpha=0.4, label='训练集')
31 plt.scatter(X_test, y_test, c='red', s=2, alpha=0.4, label='测试集')
32 # 回归线: 在 X 全量上预测, 横轴按序排列更美观
33 X_line = np.linspace(X.min(), X.max(), 200).reshape(-1, 1)
34 plt.plot(X_line, model.predict(X_line), 'g-', lw=2, label='回归线')
35 plt.xlabel('MedInc (收入中位数)'); plt.ylabel('House Value')
36 plt.legend(); plt.grid(True, alpha=0.3); plt.show()

```

Listing 1: 线性回归——完整代码

要点:

- `X[:, 0:1]` 保持二维——sklearn 要求特征矩阵必须是 (m, d) , 即使 $d = 1$ 。
- $R^2 \approx 0.47$ 说明单靠收入只能解释 47% 的房价方差, 模型欠拟合。
- `model.score()` 对回归模型自动返回 R^2 。

第二题：逻辑斯蒂分类——乳腺癌诊断（15 分）

使用 sklearn 内置的乳腺癌数据集（load_breast_cancer，二分类）。

要求写出完整代码实现以下步骤：

- (1) 加载数据，用 StandardScaler 标准化特征，划分训练/测试集（7:3，分层抽样）。（3 分）
- (2) 训练逻辑斯蒂回归模型（solver='lbfgs', max_iter=5000）。（2 分）
- (3) 预测测试集，输出混淆矩阵、Accuracy、Precision、Recall、 F_1 。（5 分）
- (4) 计算 ROC 曲线的 AUC 值，绘制 ROC 曲线。（5 分）

【参考答案】

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import load_breast_cancer
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.linear_model import LogisticRegression
7 from sklearn.metrics import (confusion_matrix, accuracy_score,
8                               precision_score, recall_score, f1_score, roc_curve, auc)
9
10 # (1) 加载 + 标准化 + 划分
11 data = load_breast_cancer()
12 X, y = data.data, data.target
13 X = StandardScaler().fit_transform(X)
14 X_train, X_test, y_train, y_test = train_test_split(
15     X, y, test_size=0.3, random_state=42, stratify=y)
16
17 # (2) 训练
18 model = LogisticRegression(solver='lbfgs', max_iter=5000)
19 model.fit(X_train, y_train)
20
21 # (3) 评估指标
22 y_pred = model.predict(X_test)
23 cm = confusion_matrix(y_test, y_pred)
24 print("混淆矩阵:\n", cm)
25 # [[ 62   1]      TN=62, FP=1
26 #   [  2 106]]    FN=2, TP=106
```

```
27
28 acc = accuracy_score(y_test, y_pred)
29 prec = precision_score(y_test, y_pred)
30 rec = recall_score(y_test, y_pred)
31 f1 = f1_score(y_test, y_pred)
32 print(f"Accuracy ={acc:.4f}")      #    0.982
33 print(f"Precision={prec:.4f}")    #    0.991
34 print(f"Recall    ={rec:.4f}")     #    0.981
35 print(f"F1        ={f1:.4f}")     #    0.986
36
37 # (4) ROC + AUC
38 y_prob = model.predict_proba(X_test)[: , 1]
39 fpr, tpr, _ = roc_curve(y_test, y_prob)
40 roc_auc = auc(fpr, tpr)
41 plt.plot(fpr, tpr, 'b-', lw=2, label=f'ROC (AUC={roc_auc:.4f})')
42 plt.plot([0, 1], [0, 1], 'k--', lw=1)
43 plt.xlabel('False Positive Rate')
44 plt.ylabel('True Positive Rate')
45 plt.title('ROC Curve - Breast Cancer')
46 plt.legend(loc='lower right'); plt.grid(True, alpha=0.3)
47 plt.show()
```

Listing 2: 逻辑斯蒂分类——完整代码

要点:

- 逻辑斯蒂对特征尺度敏感，标准化后收敛更快。
- `stratify=y` 保证训练/测试集中正负比例一致。
- 混淆矩阵: $C_{00}=TN$, $C_{01}=FP$, $C_{10}=FN$, $C_{11}=TP$ 。
- Precision ≈ 0.99 表示预测为恶性中几乎全对；Recall ≈ 0.98 表示真实恶性中 98% 被找出。

第三题：K-Means 聚类——客户分群（15 分）

使用 `make_blobs` 生成模拟客户数据：4 个簇，500 个样本，`cluster_std=1.2`。

要求写出完整代码实现以下步骤：

- (1) 生成数据，散点图展示原始分布。（2 分）
- (2) 对 $K = 1 \sim 10$ 分别聚类，记录 `inertia_`，绘制肘部曲线。（5 分）
- (3) 根据肘部曲线确定最佳 K ，输出判断依据。（3 分）
- (4) 用最佳 K 重新聚类，绘制聚类结果，质心用红色 X 标出。（5 分）

【参考答案】

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import make_blobs
4 from sklearn.cluster import KMeans
5
6 # (1) 生成数据
7 X, _ = make_blobs(n_samples=500, centers=4,
8                   cluster_std=1.2, random_state=42)
9 plt.figure(figsize=(5,4))
10 plt.scatter(X[:, 0], X[:, 1], s=10)
11 plt.title('原始数据'); plt.show()
12
13 # (2) 肘部法则：K=1..10
14 inertias = []
15 K_range = range(1, 11)
16 for k in K_range:
17     km = KMeans(n_clusters=k, random_state=42, n_init='auto')
18     km.fit(X)
19     inertias.append(km.inertia_)
20
21 # (3) 绘制肘部曲线
22 plt.figure(figsize=(7,4))
23 plt.plot(list(K_range), inertias, 'bo-', markersize=8)
24 plt.axvline(x=4, color='red', linestyle='--', label='最佳 K=4')
25 plt.xlabel('K (簇数)'); plt.ylabel('Inertia')
26 plt.title('肘部法则 (Elbow Method)')
27 plt.xticks(list(K_range)); plt.legend(); plt.grid(True, alpha=0.3)
```

```
28 plt.show()
29 print("K=4 处曲线明显变缓，为肘部拐点")
30
31 # (4) 最佳K聚类
32 best_k = 4
33 best_km = KMeans(n_clusters=best_k, random_state=42, n_init='auto')
34 labels = best_km.fit_predict(X)
35
36 plt.figure(figsize=(7,5))
37 plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=15)
38 plt.scatter(best_km.cluster_centers_[0],
39             best_km.cluster_centers_[1],
40             c='red', marker='X', s=250, edgecolors='darkred',
41             label='质心', zorder=5)
42 plt.title(f'K-Means 聚类结果 (K={best_k})')
43 plt.legend(); plt.grid(True, alpha=0.2); plt.show()
```

Listing 3: K-Means 聚类——完整代码

要点：

- `inertia_` = 每个样本到其簇质心的距离平方和。 $K \uparrow \rightarrow inertia \downarrow$ 。
- 肘部位置是边际收益骤降的拐点，对应真实簇数。
- `n_init='auto'` 让 sklearn 自动选择较优的初始质心。

第四题：朴素贝叶斯——鸢尾花分类 + 模型对比（15 分）

使用 sklearn 内置的鸢尾花数据集（load_iris, 3 分类, 4 个连续特征）。

要求写出完整代码实现以下步骤：

- (1) 加载数据，划分训练/测试集（7:3，分层抽样）。（2 分）
- (2) 用 **GaussianNB** 训练，输出测试准确率。（3 分）
- (3) 用 **MultinomialNB** 训练（提示：连续特征需先用 **MinMaxScaler** 缩放到 [0,1] 再乘 20 取整），输出测试准确率。（5 分）
- (4) 用 **5 折交叉验证** 比较两个模型，指出哪个更适合该数据集并说明原因。（5 分）

【参考答案】

```
1 import numpy as np
2 from sklearn.datasets import load_iris
3 from sklearn.model_selection import train_test_split, cross_val_score
4 from sklearn.naive_bayes import GaussianNB, MultinomialNB
5 from sklearn.preprocessing import MinMaxScaler
6
7 # (1) 加载 + 划分
8 iris = load_iris()
9 X, y = iris.data, iris.target
10 X_train, X_test, y_train, y_test = train_test_split(
11     X, y, test_size=0.3, random_state=42, stratify=y)
12
13 # (2) GaussianNB: 直接处理连续特征
14 gnb = GaussianNB()
15 gnb.fit(X_train, y_train)
16 acc_gnb = gnb.score(X_test, y_test)
17 print(f"GaussianNB 测试准确率: {acc_gnb:.4f}")    #    0.978
18
19 # (3) MultinomialNB: 连续特征需先离散化
20 scaler = MinMaxScaler()
21 X_binned = (scaler.fit_transform(X) * 20).astype(int)
22 X_train_m, X_test_m, _, _ = train_test_split(
23     X_binned, y, test_size=0.3, random_state=42, stratify=y)
24
25 mnb = MultinomialNB(alpha=1.0)
26 mnb.fit(X_train_m, y_train_m)
```



```
27 acc_mnb = mnb.score(X_test_m, y_test_m)
28 print(f"MultinomialNB 测试准确率: {acc_mnb:.4f}") # 0.933
29
30 # (4) 5折交叉验证比较
31 cv_gnb = cross_val_score(gnb, X, y, cv=5).mean()
32 cv_mnb = cross_val_score(mnb, X_binned, y, cv=5).mean()
33 print(f"\nGaussianNB 5折CV: {cv_gnb:.4f}") # 0.953
34 print(f"MultinomialNB 5折CV: {cv_mnb:.4f}") # 0.913
35 print("\n结论: GaussianNB 更优。")
36 print("原因: Iris 的特征(花萼/花瓣长宽)是连续数值, "
37       "GaussianNB 直接建模正态分布; "
38       "MultinomialNB 缩放取整会丢失精度。")
```

Listing 4: 朴素贝叶斯模型对比——完整代码

要点:

- **GaussianNB**: 假设每个特征在各类内服从正态分布, 估计 μ 和 σ^2 , 天然适配连续特征。
- **MultinomialNB**: 假设特征是离散计数 (如词频), 连续特征必须先离散化 (缩放取整), 信息有损。
- **交叉验证**比单次划分更稳定: 用 5 折平均避免某次划分的偶然性好坏。
- **分层抽样** (`stratify=y`) 保证每折中类别比例与整体一致。

第五题：Keras 全连接网络——MNIST 手写数字识别（20 分）

使用 TensorFlow/Keras 在 MNIST 上搭建三层全连接网络。

要求写出完整代码实现以下步骤：

- (1) 加载 MNIST，将 28×28 图像展平为 784 维向量，归一化到 $[0, 1]$ 。标签做 one-hot 编码（10 类）。（3 分）
- (2) 搭建 Sequential 模型： $\text{Dense}(128, \text{ReLU}) \rightarrow \text{Dropout}(0.3) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dropout}(0.3) \rightarrow \text{Dense}(10, \text{Softmax})$ 。（5 分）
- (3) 编译（Adam 优化器，categorical_crossentropy，监控 accuracy），训练 10 个 epoch，batch_size=128，validation_split=0.2。（4 分）
- (4) 评估测试集准确率。（2 分）
- (5) 绘制训练/验证的 loss 曲线和 accuracy 曲线（双子图）。（6 分）

【参考答案】

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from tensorflow import keras
4 from tensorflow.keras import layers
5
6 # (1) 加载 + 预处理
7 (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data(
8     ()
9 )
10 x_train = x_train.reshape(-1, 784).astype('float32') / 255.0
11 x_test = x_test.reshape(-1, 784).astype('float32') / 255.0
12 y_train = keras.utils.to_categorical(y_train, 10)
13 y_test = keras.utils.to_categorical(y_test, 10)
14
15 # (2) 搭建模型
16 model = keras.Sequential([
17     layers.Dense(128, activation='relu', input_shape=(784,)),
18     layers.Dropout(0.3),
19     layers.Dense(64, activation='relu'),
20     layers.Dropout(0.3),
21     layers.Dense(10, activation='softmax')
22 ])
```

```
22 # (3) 编译 + 训练
23 model.compile(optimizer='adam',
24               loss='categorical_crossentropy',
25               metrics=['accuracy'])
26 history = model.fit(x_train, y_train,
27                   batch_size=128, epochs=10,
28                   validation_split=0.2, verbose=1)
29
30 # (4) 测试评估
31 test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
32 print(f"测试准确率: {test_acc:.4f}")    #    0.977
33
34 # (5) 训练曲线
35 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
36 ax1.plot(history.history['loss'], label='训练 Loss')
37 ax1.plot(history.history['val_loss'], label='验证 Loss')
38 ax1.set_title('Loss 曲线'); ax1.set_xlabel('Epoch')
39 ax1.legend(); ax1.grid(True, alpha=0.3)
40
41 ax2.plot(history.history['accuracy'], label='训练 Accuracy')
42 ax2.plot(history.history['val_accuracy'], label='验证 Accuracy')
43 ax2.set_title('Accuracy 曲线'); ax2.set_xlabel('Epoch')
44 ax2.legend(); ax2.grid(True, alpha=0.3)
45 plt.tight_layout(); plt.show()
```

Listing 5: Keras 全连接网络——完整代码

要点:

- Keras 三步曲: Sequential(构建) → compile(编译) → fit(训练)。
- Dropout 只在训练时生效, 测试时自动关闭。
- 如果验证 loss 上升而训练 loss 持续下降 → 过拟合, 需增大 Dropout 或减 epoch。
- reshape(-1, 784) 将 (60000, 28, 28) → (60000, 784), -1 表示自动推断。

第六题：Keras CNN——CIFAR-10 图像分类（20 分）

使用 TensorFlow/Keras 在 CIFAR-10 (32×32 彩色, 10 类) 上搭建一个小型 CNN。

要求写出完整代码实现以下步骤：

(1) 加载 CIFAR-10, 归一化到 $[0, 1]$, 标签 one-hot 编码。(2 分)

(2) 搭建 CNN:

- Conv2D(32, 3×3 , ReLU, padding='same')
- MaxPooling2D(2×2)
- Conv2D(64, 3×3 , ReLU, padding='same')
- MaxPooling2D(2×2)
- Flatten $\rightarrow$ Dense(128, ReLU) $\rightarrow$ Dropout(0.5) $\rightarrow$ Dense(10, Softmax)

(7 分)

(3) 编译并训练 10 epoch (Adam, batch_size=64, validation_split=0.2)。(4 分)

(4) 评估测试集准确率。(2 分)

(5) 绘制训练/验证 accuracy 曲线。(5 分)

【参考答案】

```
1 import matplotlib.pyplot as plt
2 from tensorflow import keras
3 from tensorflow.keras import layers
4
5 # (1) 加载 + 预处理
6 (x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.
    load_data()
7 x_train = x_train.astype('float32') / 255.0
8 x_test  = x_test.astype('float32') / 255.0
9 y_train = keras.utils.to_categorical(y_train, 10)
10 y_test  = keras.utils.to_categorical(y_test, 10)
11
12 # (2) 搭建 CNN
13 model = keras.Sequential([
14     layers.Conv2D(32, (3, 3), activation='relu',
15                   padding='same', input_shape=(32, 32, 3)),
16     layers.MaxPooling2D((2, 2)),
17
```

```

18     layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
19     layers.MaxPooling2D((2, 2)),
20
21     layers.Flatten(),
22     layers.Dense(128, activation='relu'),
23     layers.Dropout(0.5),
24     layers.Dense(10, activation='softmax')
25 ])
26
27 # 查看模型结构
28 model.summary()
29
30 # (3) 编译 + 训练
31 model.compile(optimizer='adam',
32               loss='categorical_crossentropy',
33               metrics=['accuracy'])
34 history = model.fit(x_train, y_train,
35                    batch_size=64, epochs=10,
36                    validation_split=0.2, verbose=1)
37
38 # (4) 测试评估
39 test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
40 print(f"CNN 测试准确率: {test_acc:.4f}")    #    0.72
41
42 # (5) Accuracy 曲线
43 plt.figure(figsize=(7,4))
44 plt.plot(history.history['accuracy'],      'b-', label='训练 Accuracy'
45          )
46 plt.plot(history.history['val_accuracy'], 'r-', label='验证 Accuracy'
47          )
48 plt.xlabel('Epoch'); plt.ylabel('Accuracy')
49 plt.title('CNN on CIFAR-10')
50 plt.legend(); plt.grid(True, alpha=0.3); plt.show()

```

Listing 6: Keras CNN——完整代码

维度追踪（理解数据流）：

$$\underbrace{(32, 32, 3)}_{\text{输入}} \xrightarrow{\text{Conv32, pad=same}} (32, 32, 32) \xrightarrow{\text{MaxPool 2x2}} (16, 16, 32)$$

$$\xrightarrow{\text{Conv64, pad=same}} (16, 16, 64) \xrightarrow{\text{MaxPool 2x2}} (8, 8, 64) \xrightarrow{\text{Flatten}} 4096 \xrightarrow{\text{Dense}} 10$$

要点：

- padding='same' 保持空间尺寸不变；padding='valid' 会缩小。
 - MaxPool 将特征图尺寸减半（ 2×2 窗口，默认 stride=2），同时保留最显著特征。
 - CNN 参数远少于全连接：Conv 层参数 = $K_h \times K_w \times C_{in} \times C_{out} + C_{out}$ 。
 - CIFAR-10 比 MNIST 复杂得多（彩色 + 多物体），10 epoch 准确率约 72%，需更多 epoch 或更复杂架构来提升。
-
-

——试卷完，共 6 题 100 分——